Informe de Configuracion — API REST tienda_online

Construccion de una API REST de productos y categorias con Django, Django REST Framework y MySQL.

Autor: aaron03042005@gmail.com
Fecha: 18/09/2026
Proyecto: tienda_online
Base de datos: MySQL 8.0.46
Framework: Django 5.2.17 (LTS)

Contenido

1. Objetivo
2. Estado inicial y diagnostico
3. Entorno y dependencias
4. Apps instaladas (settings.py)
5. Conexion a MySQL
6. Modelos y administrador
7. Serializers, vistas y URLs
8. Pruebas de la API
9. Coleccion Postman
10. Reporte de configuracion
11. Conclusion

1. Objetivo

Instalar Python, Django y Django REST Framework; crear el proyecto **tienda_online**; configurar MySQL como motor de base de datos; construir el modelo **Product** (con categorias); exponer serializador, vistas y URLs; y validar el funcionamiento de la API en **http://127.0.0.1:8000/api/products/**.

2. Estado inicial y diagnostico

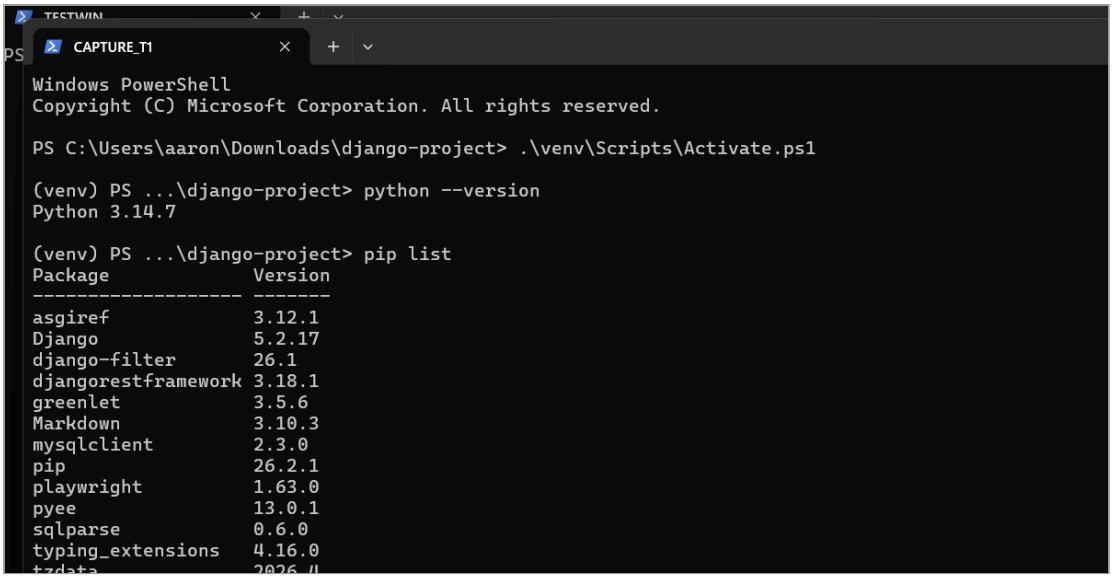
El proyecto ya tenia el esqueleto creado por **django-admin startproject tienda_online .** Al revisar **tienda_online/settings.py** se encontro el bloqueo que detuvo el avance: faltaba una coma despues de **'rest_framework'** en **INSTALLED_APPS**, lo que impedia ejecutar cualquier comando de **manage.py**.

```
INSTALLED_APPS = [
    'rest_framework'          <- falta la coma
    'django.contrib.admin',
    ...
]
```

A partir de aqui se completo el resto de la guia de principio a fin: dependencias, MySQL, modelos, serializers, vistas, URLs y pruebas.

3. Entorno y dependencias

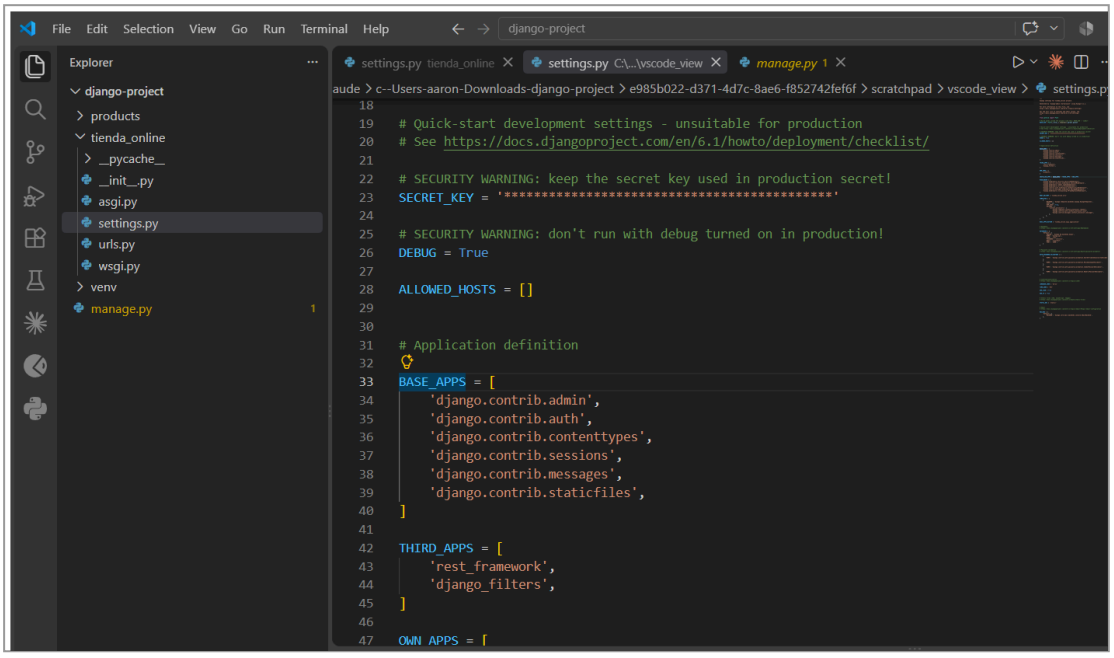
Entorno virtual activado. Se verificaron las versiones de Python y los paquetes instalados:



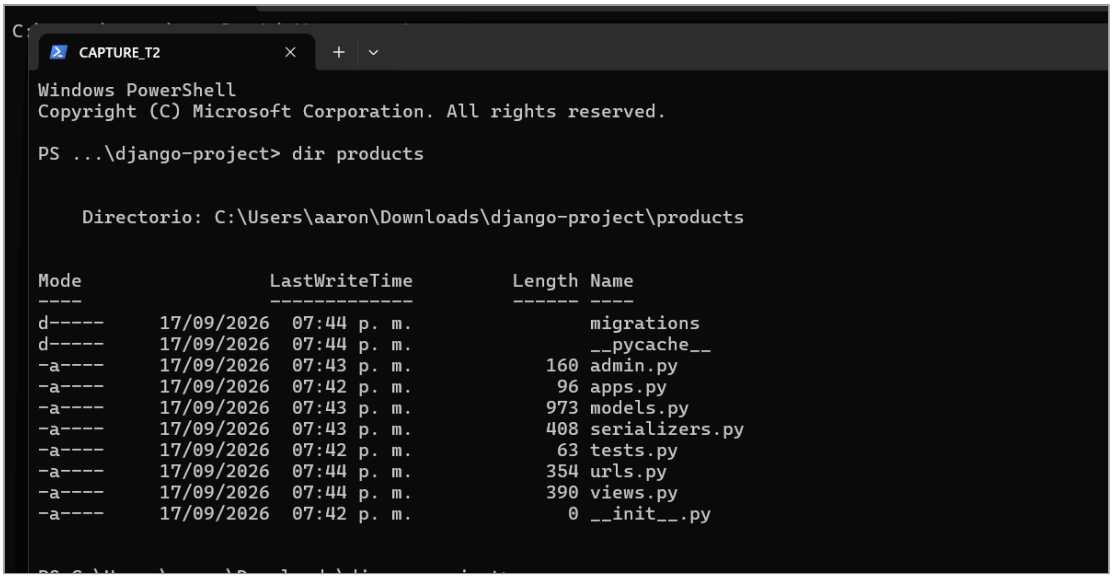
Nota de version: Django se instalo inicialmente en 6.1.1 y luego se fijo en 5.2.17 (LTS) por el incidente de compatibilidad con MySQL descrito en la seccion 5.

4. Apps instaladas (settings.py)

Se reorganizo **INSTALLED_APPS** en tres grupos (BASE_APPS, THIRD_APPS, OWN_APPS) y se corrigio la coma faltante:



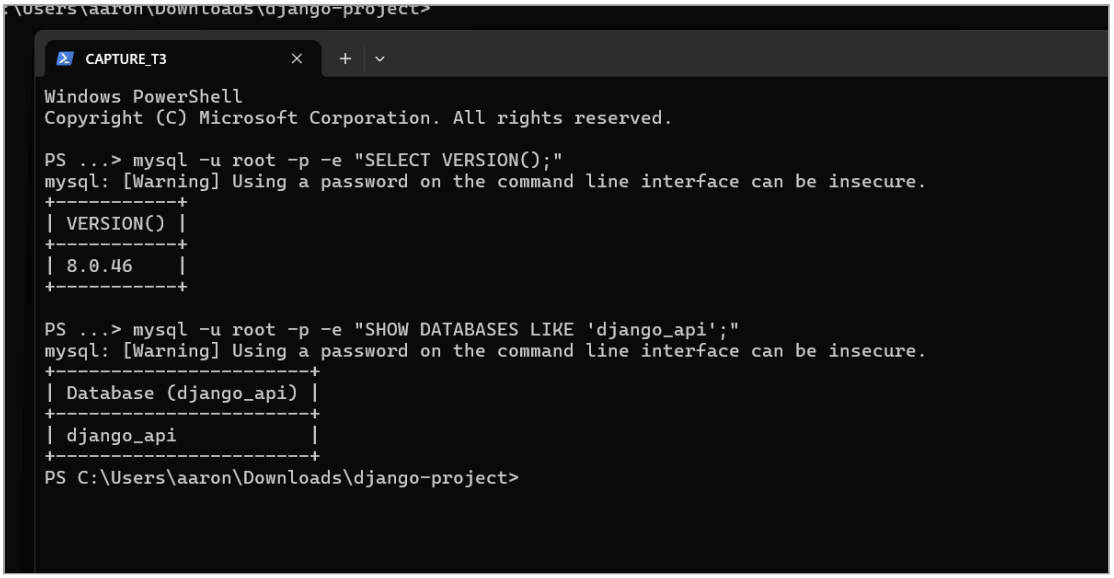
La app **products** se genero con **django-admin startapp products**:

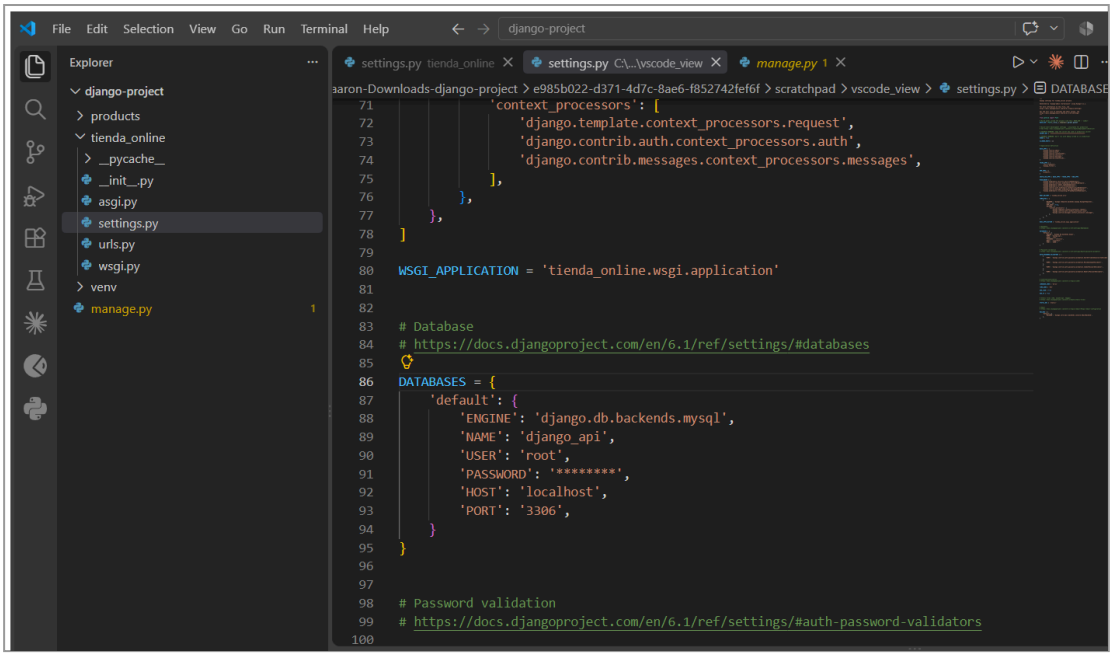


5. Conexion a MySQL

Se creo la base de datos **django_api** en el servidor MySQL local (version 8.0.46, servicio MySQL80) y se configuro Django para conectarse a ella con el usuario **root** en **localhost**, puerto 3306.

> **Incidente de compatibilidad:** Django 6.1.1 exige MySQL 8.4+, y el servidor local es 8.0.46 (**NotSupportedError: MySQL 8.4 or later is required**). Se fijo Django en la version LTS 5.2, compatible con MySQL 8.0+.



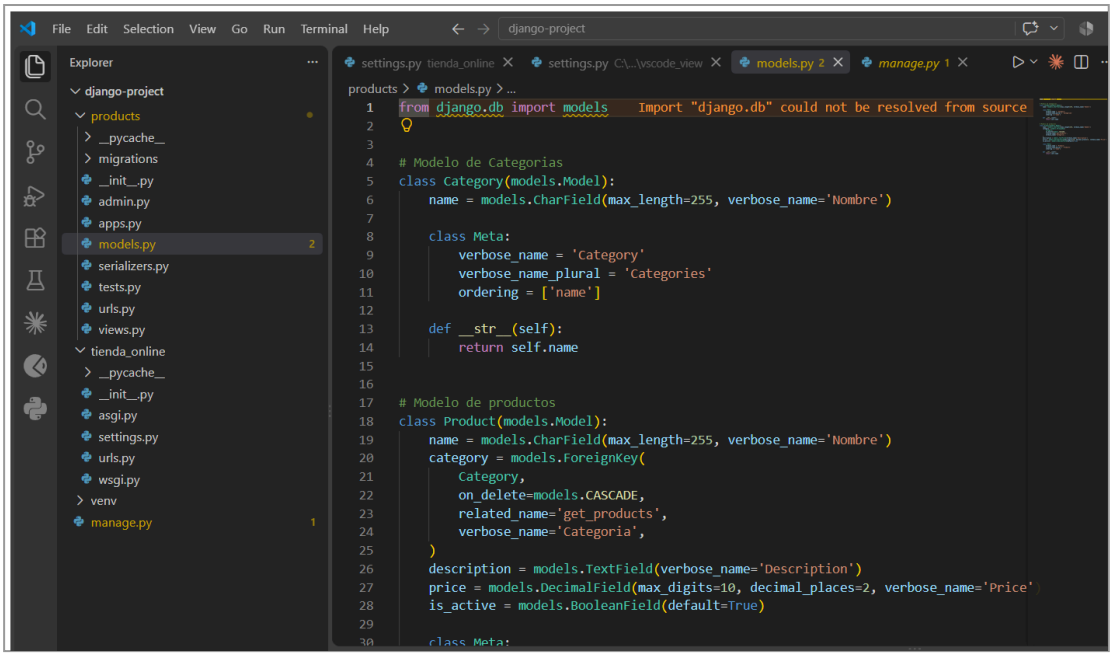


Esta ultima captura y la de INSTALLED_APPS se tomaron sobre una copia temporal de **settings.py** con **SECRET_KEY** y la contrasena de MySQL sustituidas por asteriscos, para no exponer credenciales reales en este documento.

6. Modelos y administrador

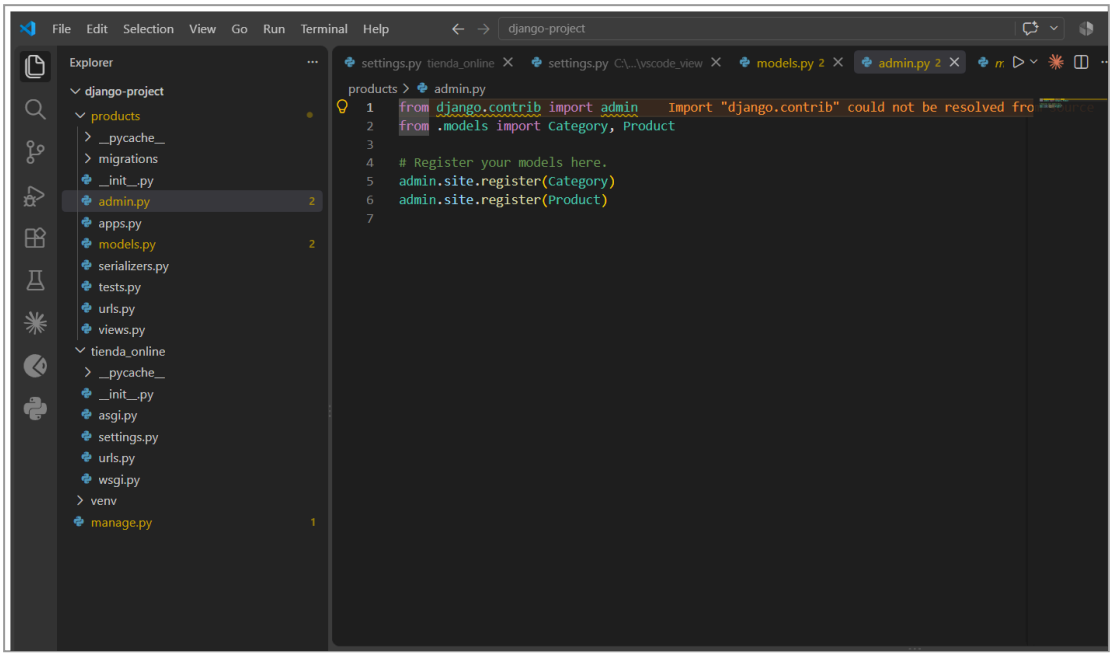
```
class Category(models.Model):
    name = models.CharField(max_length=255, verbose_name='Nombre')
    class Meta:
        verbose_name = 'Category'
        verbose_name_plural = 'Categories'
        ordering = ['name']
    def __str__(self):
        return self.name

class Product(models.Model):
    name = models.CharField(max_length=255, verbose_name='Nombre')
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
                                 related_name='get_products',
verbose_name='Categoria')
    description = models.TextField(verbose_name='Description')
    price = models.DecimalField(max_digits=10, decimal_places=2,
verbose_name='Price')
    is_active = models.BooleanField(default=True)
```

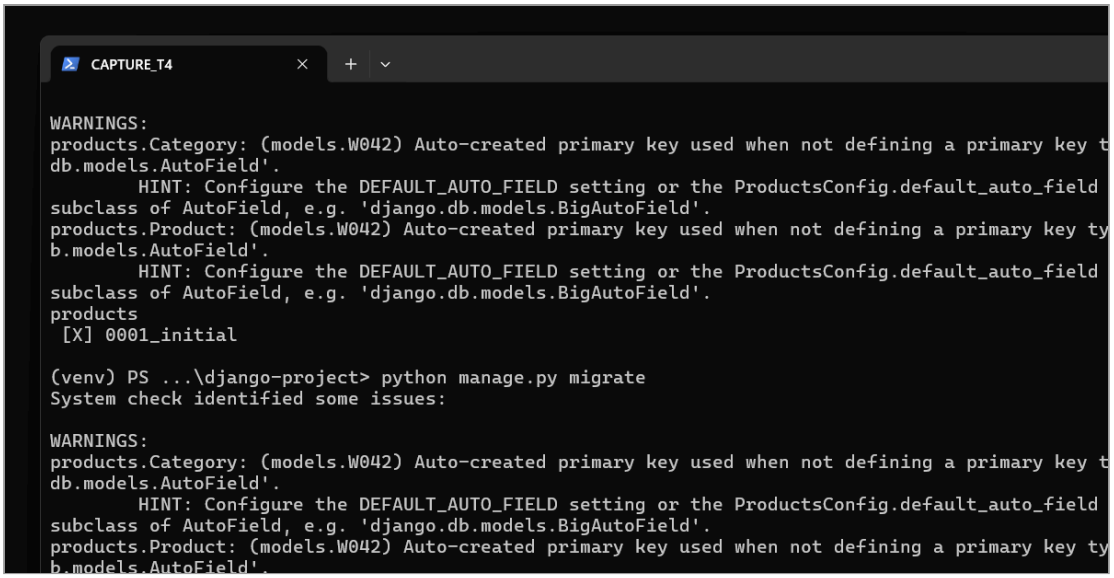


from django.contrib import admin
from .models import Category, Product

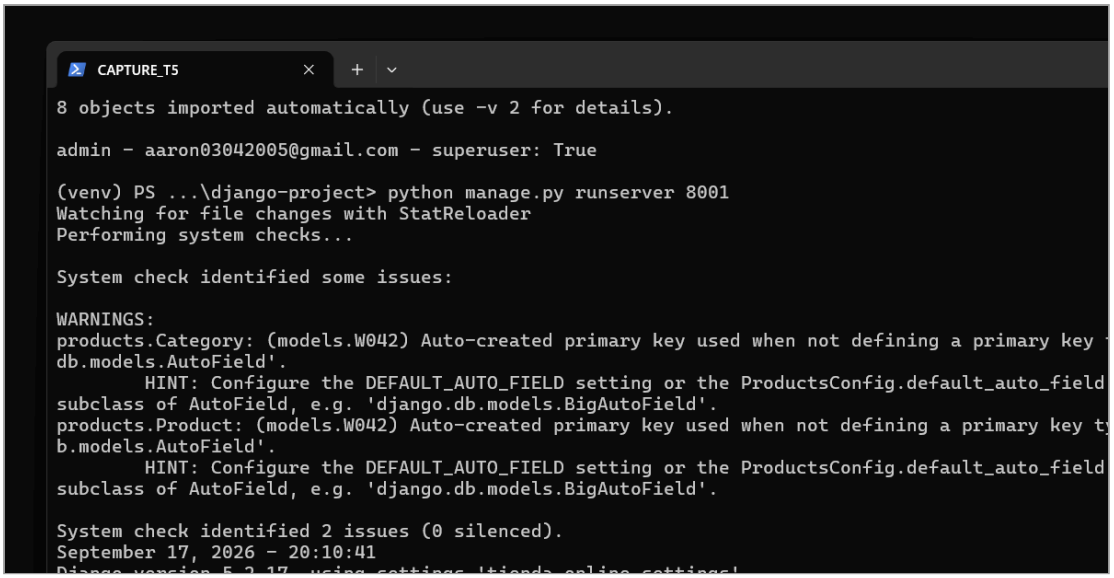
admin.site.register(Category)
admin.site.register(Product)

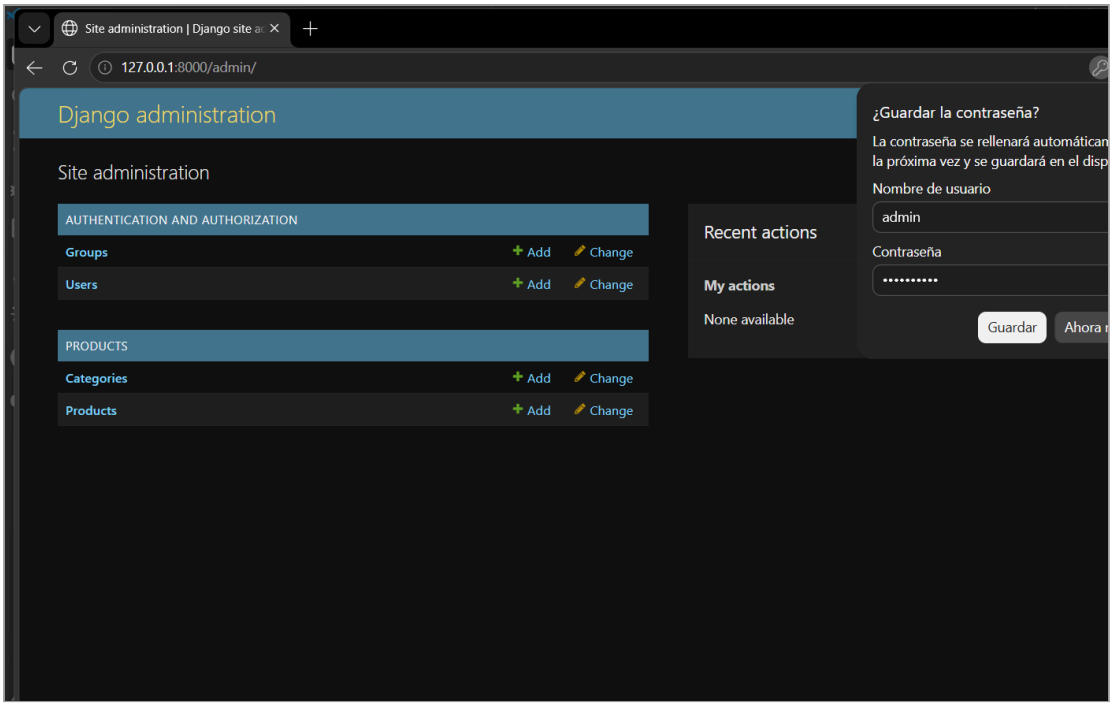


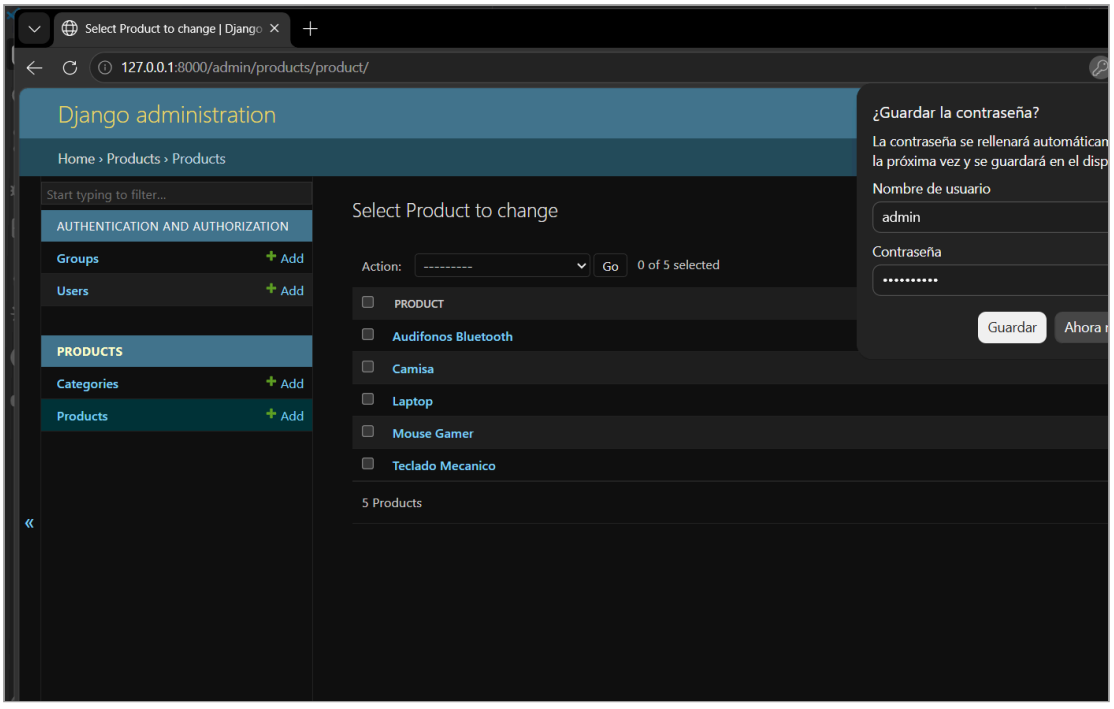
Migraciones:



Superusuario creado y arranque del servidor:



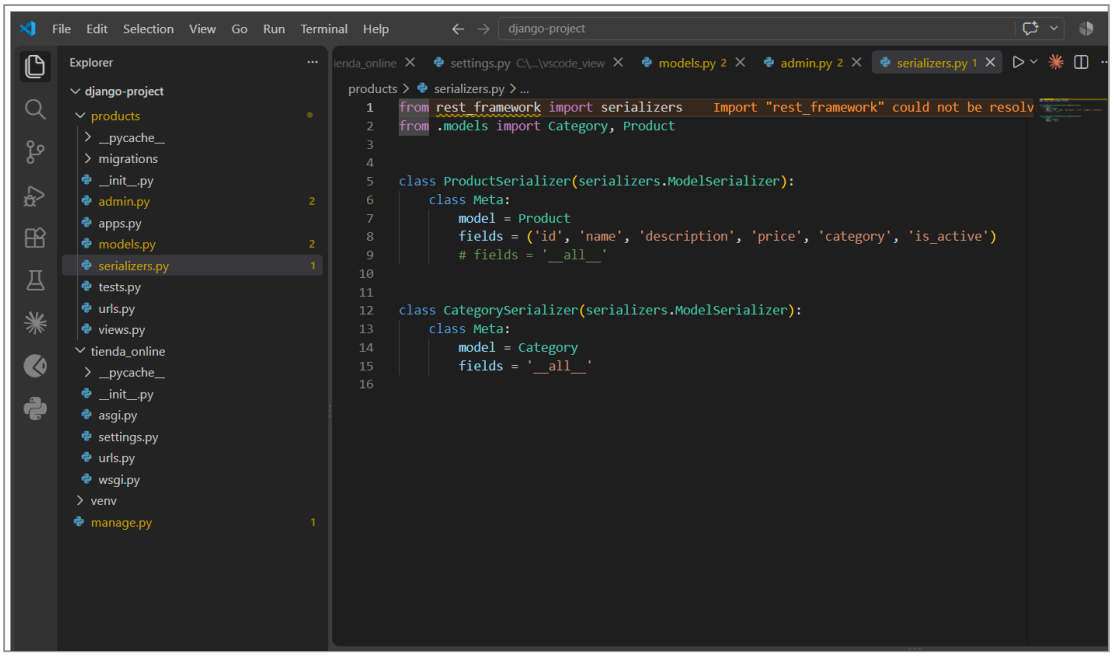

Site administration | Django site a
127.0.0.1:8000/admin/
Django administration
Site administration
AUTHENTICATION AND AUTHORIZATION
Groups + Add Change
Users + Add Change
PRODUCTS
Categories + Add Change
Products + Add Change
Recent actions
My actions
None available
¿Guardar la contraseña?
La contraseña se rellenará automáticam
la próxima vez y se guardará en el disp
Nombre de usuario
admin
Contraseña
Guardar
Ahora


Select Product to change | Django
127.0.0.1:8000/admin/products/product/
Django administration
Home › Products › Products
Start typing to filter...
AUTHENTICATION AND AUTHORIZATION
Groups + Add
Users + Add
PRODUCTS
Categories + Add
Products + Add
Select Product to change
Action: --------- Go 0 of 5 selected
PRODUCT
Audifonos Bluetooth
Camisa
Laptop
Mouse Gamer
Teclado Mecanico
5 Products
¿Guardar la contraseña?
La contraseña se rellenará automáticam
la próxima vez y se guardará en el disp
Nombre de usuario
admin
Contraseña
Guardar
Ahora

7. Serializers, vistas y URLs

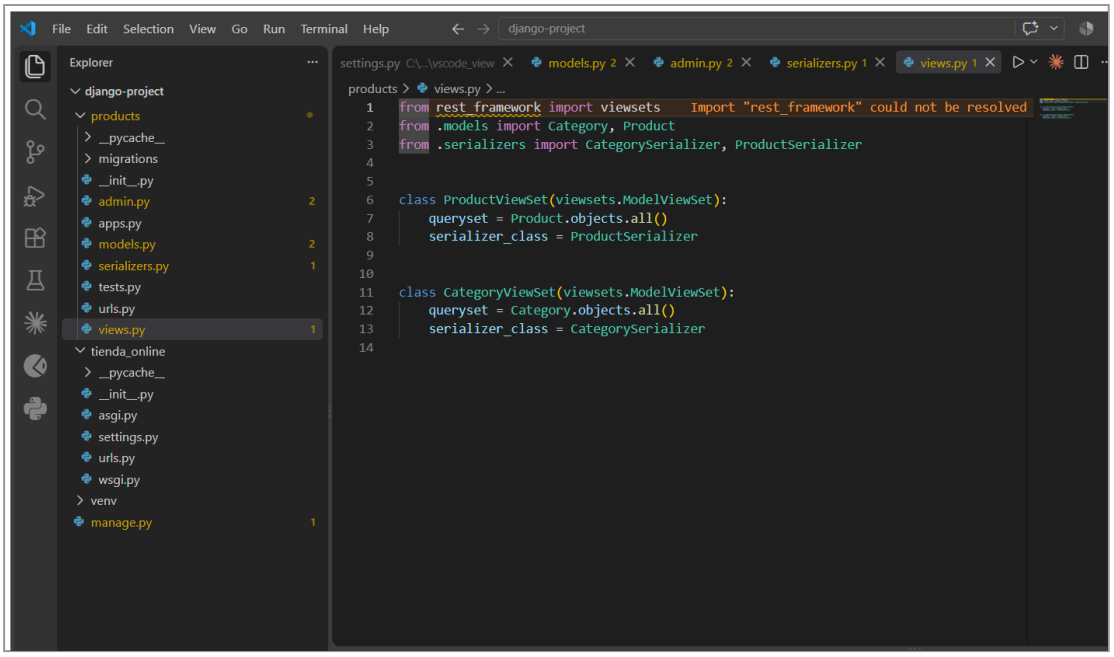
```
class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ('id', 'name', 'description', 'price', 'category', 'is_active')

class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = '__all__'
```



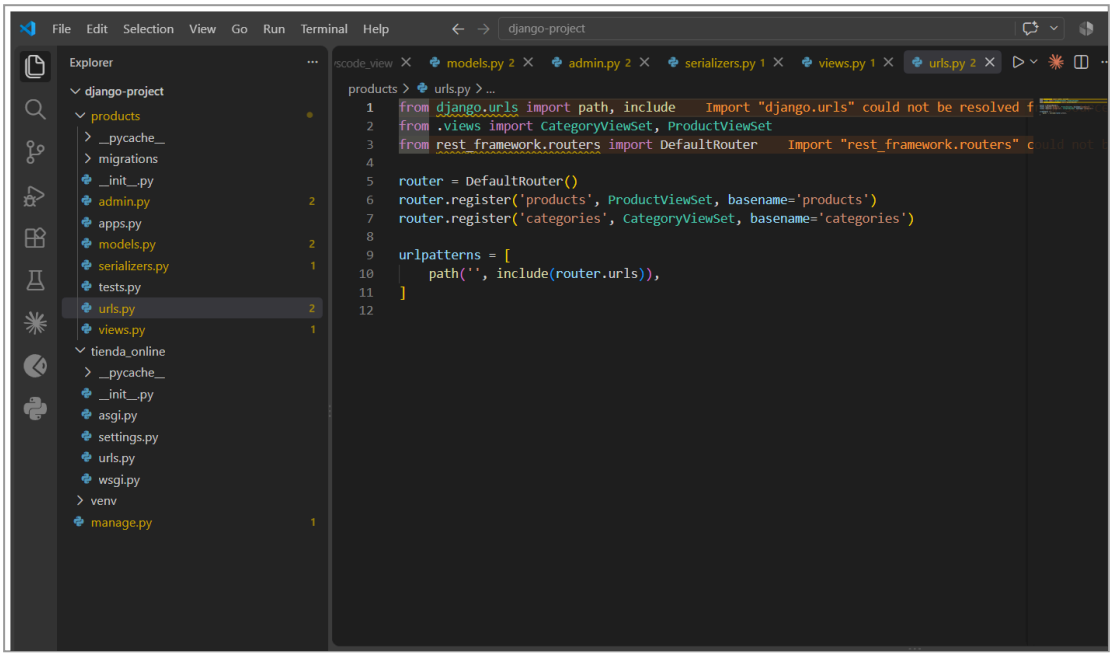

```
class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer

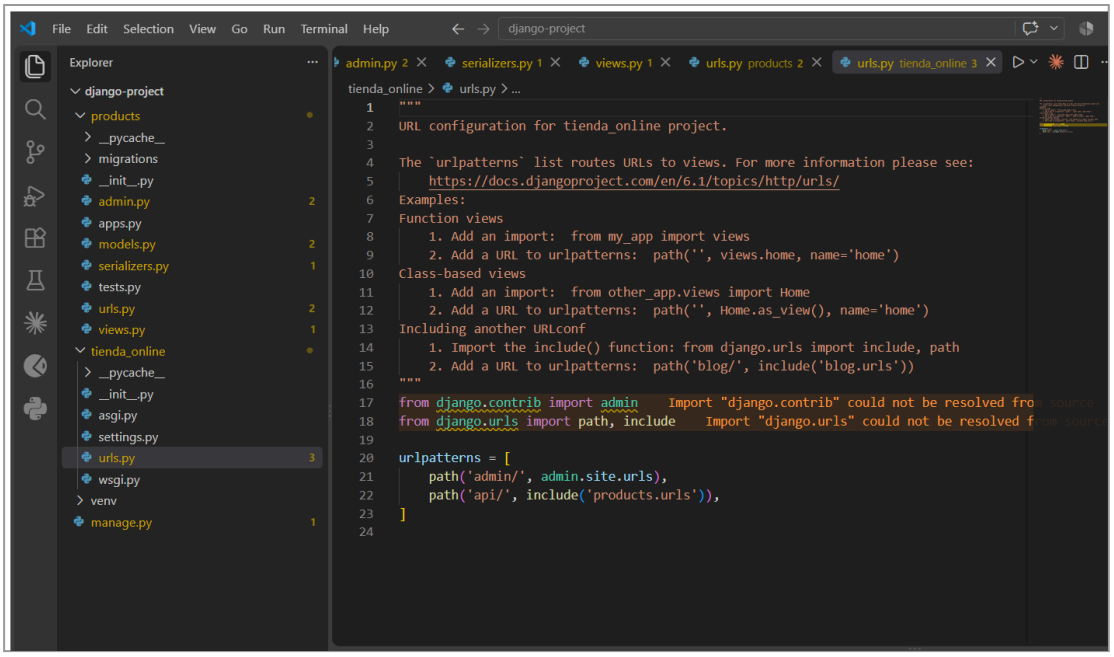
class CategoryViewSet(viewsets.ModelViewSet):
    queryset = Category.objects.all()
    serializer_class = CategorySerializer
```




```
# products/urls.py
router = DefaultRouter()
router.register('products', ProductViewSet, basename='products')
router.register('categories', CategoryViewSet, basename='categories')
urlpatterns = [ path('', include(router.urls)) ]

# tienda_online/urls.py
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('products.urls')),
]
```

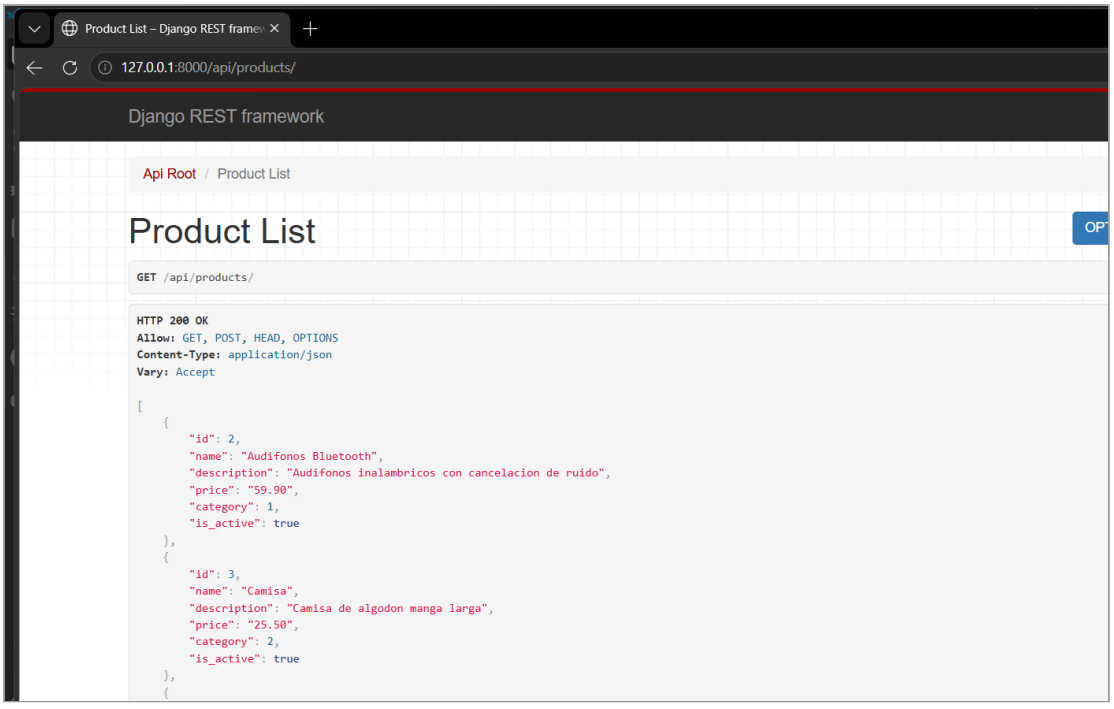


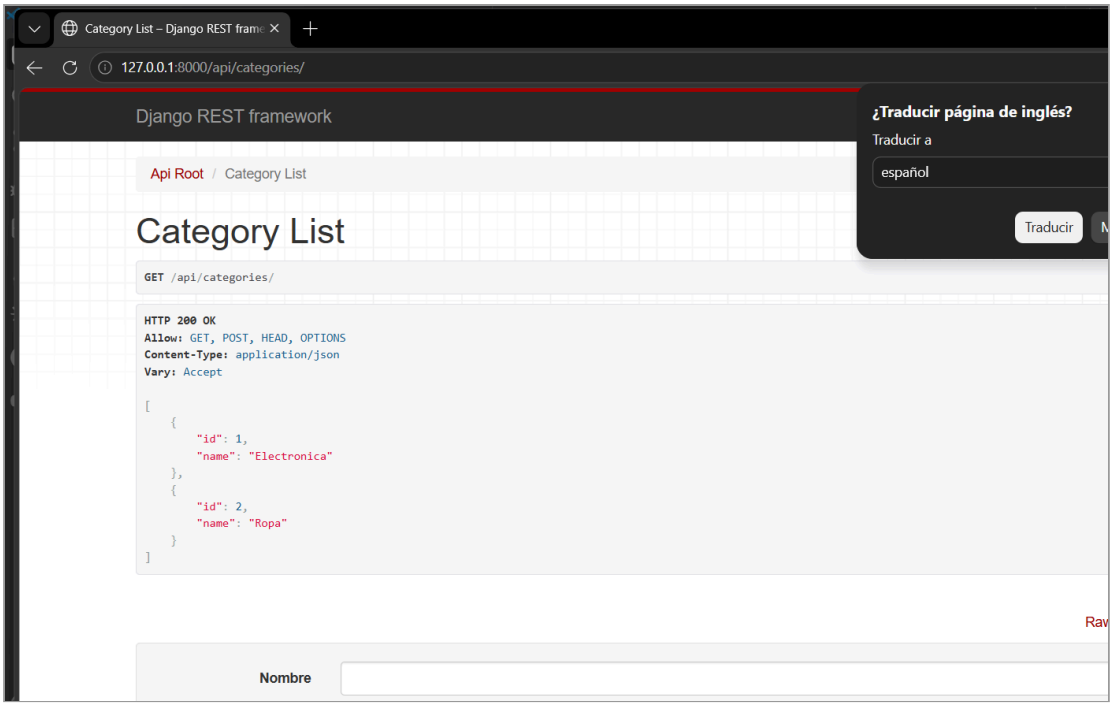


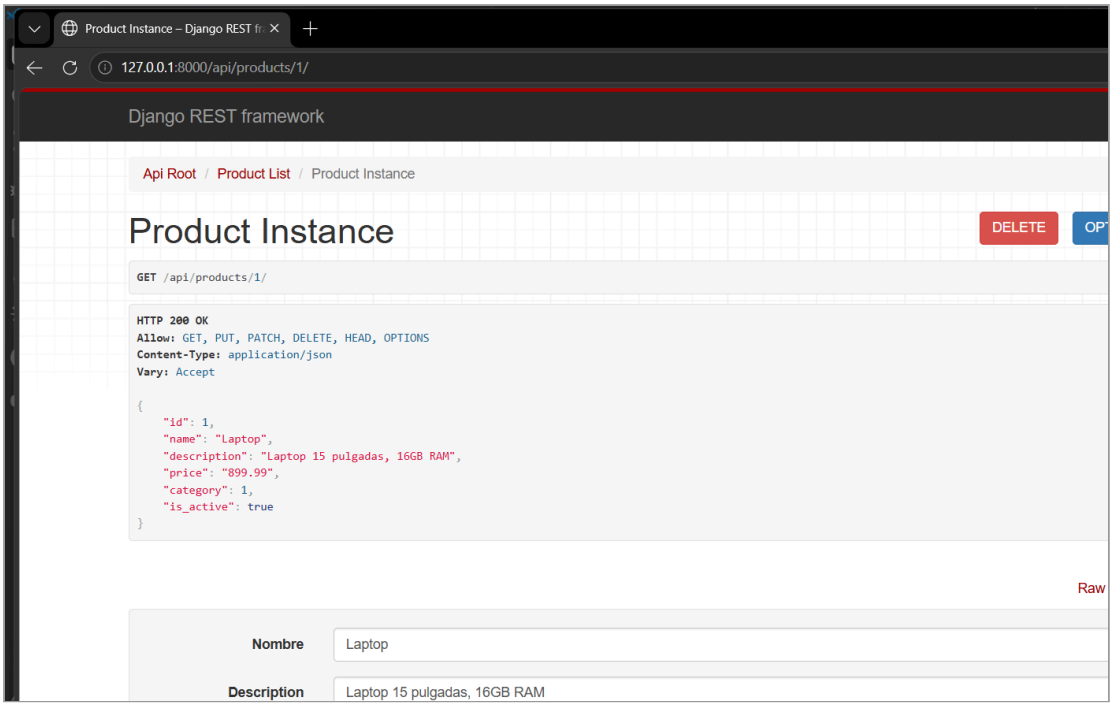
Con el router, los endpoints quedan expuestos en **/api/products/** y **/api/categories/**.

8. Pruebas de la API

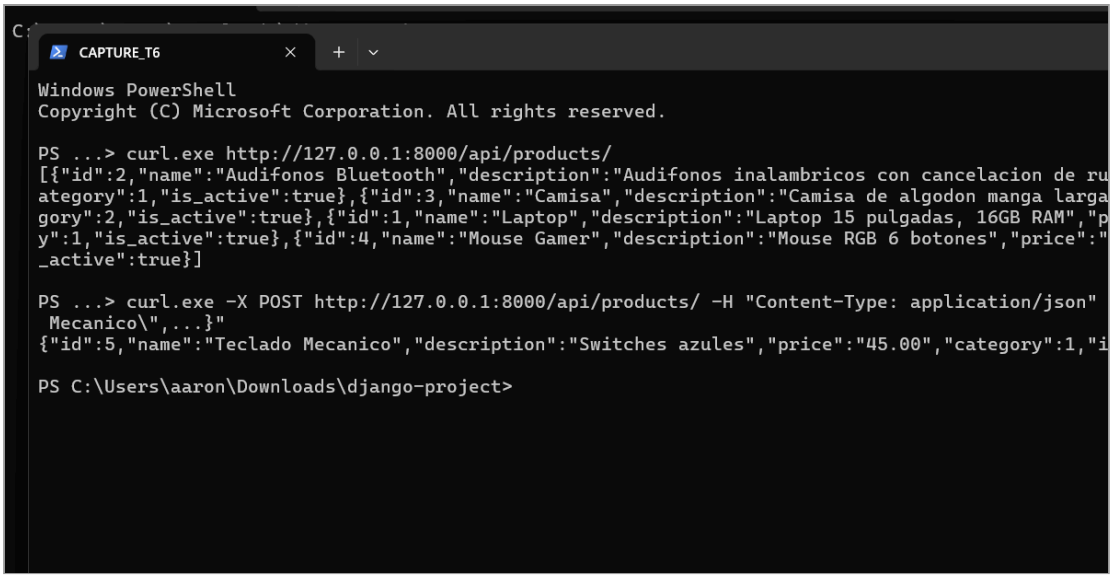
Interfaz navegable de Django REST Framework en **http://127.0.0.1:8000/api/products/**:







Verificacion por linea de comandos (GET, POST y detalle):



Se probaron los endpoints principales: pedir la lista de productos respondio correctamente (200 OK), crear un producto nuevo por POST tambien funciono (201 Created), pedir un producto especifico por su id respondio bien (200 OK), lo mismo con la lista de categorias, y el panel de administracion respondio 200 OK despues de iniciar sesion.

9. Coleccion Postman

Coleccion minima para completar la validacion en Postman:

```
{
  "info": { "name": "tienda_online API", "schema":
    "https://schema.getpostman.com/json/collection/v2.1.0/collection.json" },
  "item": [
    { "name": "List products", "request": { "method": "GET",
      "url": "http://127.0.0.1:8000/api/products/" } },
    { "name": "Create product", "request": { "method": "POST",
      "url": "http://127.0.0.1:8000/api/products/",
      "header": [{ "key": "Content-Type", "value": "application/json" }],
      "body": { "mode": "raw", "raw":
        "
{\"name\":\"Teclado\",\"description\":\"Mecanico\",\"price\":\"45.00\",\"category\"
:1}" } } },
    { "name": "List categories", "request": { "method": "GET",
      "url": "http://127.0.0.1:8000/api/categories/" } }
  ]
}
```

10. Reporte de configuracion

El proyecto quedo corriendo con Python 3.14.7, Django 5.2.17 (LTS), Django REST Framework 3.18.1, django-filter 26.1, Markdown 3.10.3 y mysqlclient 2.3.0, conectado a un servidor MySQL 8.0.46 (servicio MySQL80). El proyecto quedo en modo desarrollo (**DEBUG = True**), con **INSTALLED_APPS** organizado en tres grupos: apps base de Django, apps de terceros (**rest_framework**, **django_filters**) y la app propia **products**.

Django muestra una advertencia no bloqueante (**models.W042**) que sugiere fijar **DEFAULT_AUTO_FIELD** en **settings.py**. No afecta el funcionamiento de la API.

11. Conclusion

La API quedo completa y verificada de extremo a extremo: entorno virtual con Django 5.2 LTS y DRF, base de datos MySQL **django_api** conectada y migrada, modelos **Product**/**Category** con relacion por clave foranea, panel de administracion funcional, y endpoints REST respondiendo correctamente a GET, POST y detalle por id.

Todas las capturas de este informe se tomaron directamente del equipo donde se desarrollo el proyecto (VS Code, terminal PowerShell y navegador Edge reales).